# Day 12 - TypeScript Deep Dive + Testing with Jest

## Day 12 — TypeScript Deep Dive + Testing with Jest

## Objectives

- Learn `interface` and `type` for describing object shapes, without repeating yourself
- Learn generics — writing code that works with multiple types while still being type-checked
- Understand union types and how TypeScript handles "this could be one of several types"
- Write real tests using Jest, the most widely-used JavaScript/TypeScript testing framework

## `interface` — naming a shape, once, and reusing it everywhere

Yesterday you annotated an object's shape inline: `{ name: string; age: number }`. Once you use that same shape more than once, naming it with an `interface` avoids repeating yourself:

```
interface Person {
    name: string;
    age: number;
    email?: string;    // the ? marks this property as OPTIONAL -- it may or may not be present
}

function greet(person: Person): string {
    return `${person.name} is ${person.age}`;
}

const ana: Person = { name: "Ana", age: 30 };            // valid -- email is optional
const bo: Person = { name: "Bo", age: 25, email: "bo@example.com" };   // also valid
const bad: Person = { name: "Cy" };                            // ERROR -- missing required propert
```

An `interface` describes the *shape* a value must have — which properties it needs, and what type each one must be — without describing any actual behavior (that's what a `class`, from Days 8-9, is for). Any object matching that shape can be used wherever the interface is expected, regardless of how it was created — this is sometimes called "structural typing": TypeScript cares about the *shape* of your data, not what you formally declared it to be.

## `type` — a more general alternative to `interface`

```
type ID = string | number;    // a UNION type -- explained below
type Point = { x: number; y: number };   // this looks a lot like an interface!
```

For plain object shapes, `interface` and `type` are largely interchangeable, and you'll see both conventions in real codebases (a common guideline: prefer `interface` for object shapes, since it can be extended and gives clearer error messages; use `type` for anything else — unions, combinations of types, and shapes that aren't

plain objects at all).

## Union types — "this could be one of several types"

```
function formatId(id: string | number): string {
    return `ID-${id}`;
}

formatId(42);        // fine -- 42 is a number
formatId("abc");        // fine -- "abc" is a string
formatId(true);           // ERROR -- boolean is not part of the union string | number
```

`string | number` means "accept EITHER a string OR a number, nothing else." This is TypeScript's precise, checked way of expressing something JavaScript does constantly and loosely (a function that happens to work with either type) — with the union type, the compiler ensures every code path only does things valid for both possibilities, or forces you to explicitly check which one you actually have first:

```
function double(value: string | number): string | number {
    if (typeof value === "number") {
        return value * 2;         // TypeScript KNOWS value is a number here, thanks to the typeof c
    }
    return value + value;            // and here, TypeScript knows it must be a string
}
```

This `typeof value === "number"` check is called **narrowing**: TypeScript tracks, line by line, which specific type within a union is actually possible at each point, based on checks like this — inside the `if` block, only `number` is possible; after it, only `string` remains. This is a genuinely elegant piece of TypeScript's design, worth appreciating once it clicks.

## Generics — writing one function/type that works with many types, still fully checked

Imagine writing a function that just returns whatever it's given, unchanged — its logic doesn't care about the specific type at all:

```
function identity(value: any): any {    // works, but loses ALL type information -- `any` disables c
    return value;
}

const result = identity(5);   // TypeScript thinks `result` is `any` -- it has NO idea it's actually
```

**Generics** solve this properly — they let a function (or a type, or a class) be written once, work with any type, and still have that specific type tracked and checked correctly:

```
function identity<T>(value: T): T {   // T is a GENERIC TYPE PARAMETER -- a placeholder for "whatev
    return value;
}

const numberResult = identity(5);          // TypeScript correctly infers T is `number` here -- numb
const stringResult = identity("hello");       // and here, T is `string` -- stringResult is typed as
```

Read `<T>` as "this function works with some type, call it `T`, to be determined by whatever's actually passed in each time it's called" — and crucially, TypeScript remembers exactly which type `T` was for each individual call,

keeping full type-safety, unlike the `any` version above which throws that information away entirely.

You've actually already used a built-in generic type without necessarily noticing: `Promise<string>` (or `Promise<any>`, from yesterday) — the `<...>` there is exactly this same generic syntax, saying "a Promise that will eventually resolve with specifically a string" (or whatever type is inside the angle brackets).

A common, very practical use of generics: a reusable data structure or utility function that shouldn't care what type of data it holds:

```
function firstElement<T>(array: T[]): T | undefined {
    return array[0];
}

firstElement([1, 2, 3]);           // typed as number | undefined
firstElement(["a", "b"]);             // typed as string | undefined
```

# Setting up Jest for testing

```
npm install --save-dev jest ts-jest @types/jest @types/node
```

(This has already been installed for you at the root of this whole `typescript-javascript` project, so every day from here on can use it without reinstalling.) **Jest** is the most widely-used JavaScript/TypeScript testing framework — its role and mental model are the direct parallel of the Python track's `pytest`, if you've done that track: it automatically finds and runs your tests, and reports pass/fail with helpful detail on failures.

Jest looks for files ending in `.test.js` or `.test.ts` (or inside a `__tests__` folder). Inside them, you write tests using `test()` (or its alias `it()`) and `expect()`:

```
// mathUtils.test.ts
function add(a: number, b: number): number {
    return a + b;
}

test("adds two positive numbers", () => {
    expect(add(2, 3)).toBe(5);
});

test("adds negative numbers", () => {
    expect(add(-1, -1)).toBe(-2);
});
```

`expect(actualValue).toBe(expectedValue)` is Jest's core assertion — `.toBe()` checks strict equality (`===`), the same rule you learned Day 1. Jest has many other "matchers" beyond `.toBe()`: `.toEqual()` (checks deep equality of objects/arrays, since two different array objects with identical contents aren't `===` equal — recall Day 1's `is`/`==` distinction, or the Python track's `id()`/`==`, if you've seen either), `.toBeNull()`, `.toContain()` (does an array/string contain something), and `.toThrow()` (does calling a function throw an error):

```
function divide(a: number, b: number): number {
    if (b === 0) {
        throw new Error("cannot divide by zero");
    }
    return a / b;
}

test("divide by zero throws", () => {
```

```
    expect(() => divide(10, 0)).toThrow("cannot divide by zero");
});
```

Notice `expect(() => divide(10, 0))` — you must wrap the risky call in an extra arrow function when testing for a thrown error; if you called `divide(10, 0)` directly inside `expect(...)`, it would throw immediately, before Jest ever gets a chance to catch and check it properly.

### Testing async code

```
test("delay resolves with the given value", async () => {
    const result = await delay(10, "hello");
    expect(result).toBe("hello");
});
```

Since Jest test functions can themselves be `async` (exactly like any other function, from yesterday's lesson), testing Promise-based/async code is straightforward — just `await` inside the test, exactly as you would anywhere else.

Run your tests with:

```
npx jest
npx jest --verbose      # shows every individual test's name, not just a summary
```

## Exercises

Open `calculator.ts` and read it — it has type annotations already in place. Then open `calculator.test.ts`, fill in each `// TODO` test, and run `npx jest` from this folder.